6-2 · Docker 기반 Kafka 클러스터 · 실습 2.5H

토픽 관리와 메시지 흐름·컨슈머 그룹 검증

8단계 — 기동부터 리밸런싱까지 터미널 넷으로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

2.5시간 뒤에는

① docker compose 로 KRaft 브로커를 띄우고 두 경로로 접속한다

② orders 토픽(파티션 3)을 만들고 retention.ms 를 바꿨다 되돌린다

③ 같은 키가 같은 파티션으로 가는 것과 LAG 을 확인한다

④ 컨슈머를 늘리고 끄며 분배와 리밸런싱을 관찰한다

터미널 4개를 씁니다 — 1·4 는 컨슈머, 2 는 프로듀서, 3 은 그룹 확인용

실습 흐름

여덟 단계 — 기동부터 다른 그룹 확인까지

~/data-eng/week6 폴더의 docker-compose.yml 하나로 — 6-4·6-5 에 서비스가 계속 붙는 이번 주의 뼈대입니다

1. compose 기동	2. 두 경로 접속	3. 토픽·설정	4·5. 메시지·LAG	6~8. 그룹 관찰
config → up -d → ps	29092 · 9092 + 일부러 실패	orders 3파티션 retention.ms	키별 파티션 describe --group	분배·리밸런싱 다른 그룹

1단계

compose 의 핵심 줄 — 리스너 세 개와 복제 수 1

compose 의 줄	뜻
image: apache/kafka:3.7.0	6-1 과 같은 이미지
ports: "9092:9092"	호스트용 EXTERNAL 만 밖으로
KAFKA_LISTENERS	PLAINTEXT :29092 · CONTROLLER :29093 · EXTERNAL :9092
KAFKA_ADVERTISED_LISTENERS	PLAINTEXT kafka:29092 · EXTERNAL localhost:9092
LISTENER_SECURITY_PROTOCOL_MAP	세 리스너 모두 PLAINTEXT
*_REPLICATION_FACTOR · MIN_ISR	세 설정 모두 1 — 단일 브로커

파일은 cat > docker-compose.yml << 'EOF' 로 · Conflict "/kafka" 가 나오면 docker rm -fv kafka

config 세 줄 → up -d → ps 의 STATUS Up … · 몇 초 뒤 docker logs kafka 에 「Kafka Server started」(실측 5초)

2·3단계

핵심 명령 — 안내 확인 · 두 경로 접속 · 토픽과 설정

안내 확인	docker exec kafka env | grep -E "LISTENER|ADVERTISED"
안에서 붙기	kafka-broker-api-versions.sh --bootstrap-server kafka:29092
밖에서 붙기	docker run --rm --network host … --bootstrap-server localhost:9092
토픽 생성	kafka-topics.sh --create --topic orders --partitions 3
보존 변경	kafka-configs.sh --alter --add-config retention.ms=3600000
되돌리기	kafka-configs.sh --alter --delete-config retention.ms

두 경로 모두 브로커 정보를 돌려주면 리스너 2계층 성공 — 한쪽만 되면 ADVERTISED_LISTENERS 를 다시 본다

2단계 — 일부러 실패

컨테이너 안에서 바깥용 주소 kafka:9092 로 붙으면

1. **--network week6_default --bootstrap-server kafka:9092**
   docker run 으로 다른 컨테이너에서 — 첫 접속은 된다

2. **브로커의 안내 — 「다시 붙을 주소는 localhost:9092」**
   이 컨테이너 안의 localhost 에는 브로커가 없다

3. **Connection to node 1 ... could not be established**
   (localhost/127.0.0.1:9092) — WARN 이 계속 찍힌다

4. **Ctrl+C 로 멈춘다**
   「첫 통화는 되는데 실제 업무 통화가 안 되는」 상태 — 실측

> **컨테이너끼리는 kafka:29092 — 6-4 의 Connect 가 그렇게 붙는다**

4단계

메시지를 보내고 파티션 분배를 본다 — 같은 키는 같은 파티션

터미널 1 — Consumer

먼저 켠다 — 새 그룹은 켠 뒤 온 것만 읽는다

- kafka-console-consumer.sh
- --topic orders --group order-app
- --property print.key=true
- --property print.partition=true

VS

터미널 2 — Producer

실측 user1→0, user2·user3→2, 1번 빈 칸

- kafka-console-producer.sh
- --topic orders
- --property parse.key=true
- --property key.separator=:

5단계 — 일부러 끄기

LAG 을 읽고, 컨슈머를 꺼서 쌓이게 한다

1. **kafka-consumer-groups.sh --describe --group order-app**
   파티션별 CURRENT-OFFSET / LOG-END-OFFSET / LAG — 다 읽었으면 LAG 0

2. **터미널 1 컨슈머를 Ctrl+C 로 끈다**
   터미널 2 에서 메시지를 몇 개 더 보내고 다시 describe

3. **LAG 이 0 보다 커진다**
   실측 세 건 더 보내니 0번 LAG 1 · 2번 LAG 2 — 다시 켜면 0

운영에서 LAG 을 감시하는 이유

읽는 쪽 멈춤

= LAG 증가

6·7단계 — 일부러 죽이기

컨슈머를 둘 붙였다가 하나를 끈다 — 분배와 리밸런싱

이론의 「파티션을 나눠 갖는다」와 「컨슈머가 죽어도 처리가 멈추지 않는다」를 직접 겪어 봅니다

터미널 4 추가	describe	메시지 전송	하나를 Ctrl+C	리밸런싱
같은 그룹 order-app	CONSUMER-ID 둘 0·1번 / 2번	0·1번 쪽은 user1 것만	몇 초 뒤 다시 describe	남은 쪽이 3개 모두

8단계

다른 그룹은 독립적이다 — 그룹마다 오프셋을 따로 관리

order-app

이미 읽은 메시지는 다시 오지 않는다

- 그룹의 오프셋이 저장돼 있다
- 다시 붙으면 이어서 읽는다

VS

analytics-app --from-beginning

처음부터 다시 받는다

- 새 그룹 이름 = 새 오프셋 · 실측 일곱 건
- 6-1 의 「여러 부서가 같은 편지를 각자 읽는다」의 실물

확인 포인트

이 2.5시간이 끝날 때 확인할 것

1. docker compose ps 의 STATUS 가 Up … 이다
2. kafka:29092 · localhost:9092 두 경로 모두 브로커 정보를 돌려준다
3. describe 에 retention.ms=3600000 이 보였다가 되돌렸다
4. 같은 키는 같은 칸 — user1 은 0, user2 · user3 은 2
5. 컨슈머를 끄면 LAG 이 커지고, 둘을 붙이면 0·1번 / 2번
6. 하나를 끄면 남은 쪽이 3개를 넘겨받고, analytics-app 은 처음부터 받는다

리밸런싱 중엔 소비가 잠시 멈춥니다 — 자주 반복되면 「리밸런싱 폭풍」

6-2 · 실습 2.5H 마무리

다음은 정리·회고 (0.5H)

compose 설정 묶음을 「틀리면 무슨 일이」로 되짚고, 6-3 에서 파이썬으로 붙을 준비를 확인합니다

- ✓ **compose 설정 묶음 여섯 개 리뷰**
- ✓ **CLI 명령 요약 한 장**
- ✓ **자주 겪는 오류 다섯 가지**
- ✓ **nc -z localhost 9092 · confluent-kafka 설치**

가져갈 것

user1·2·3 이 간 파티션 번호 메모